

Agentic AI

Naeemullah Khan

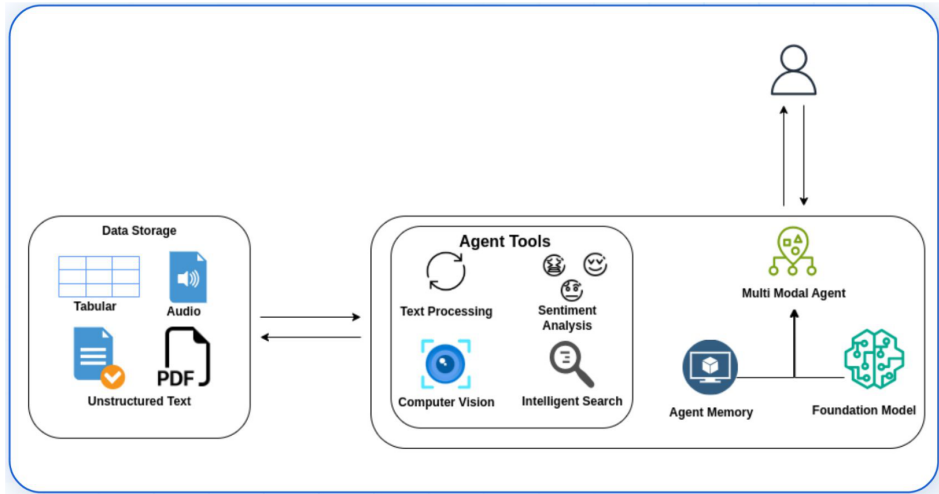
naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

August 10, 2026



Redrawn after Nath et al., “Generative AI and multi-modal agents in AWS: The key to unlocking new value in financial markets”, AWS Machine Learning Blog, 2023 (“Architecture Pattern: Multi-modal Agent”).

1. Motivation
2. Learning Outcomes
3. Agentic AI: Definitions, Reactive vs. Proactive Agents
4. Agent Loop, Tool Use, and Agentic RAG
5. Open-Source Agentic Frameworks
 - 5.1 A Taxonomy of Agent Frameworks
 - 5.2 Auto-GPT and BabyAGI
 - 5.3 Other Notable Frameworks

6. Code Generation

6.1 Benchmarks and Metrics

6.2 Methods: Context, Infilling, Retrieval, Execution Feedback

6.3 Representative Code LMs

7. Continual Learning in Agents

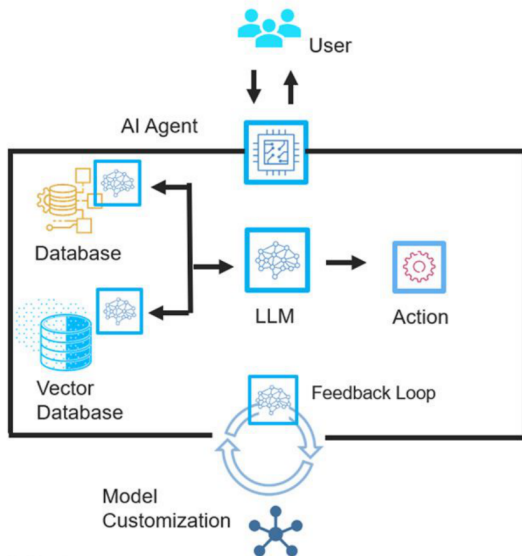
8. Safety, Ethics, and Control

9. Future Directions

- ▶ A language model answers; an **agent** acts: it pursues goals over multiple steps, using tools, memory, and feedback from its environment.
- ▶ Modern AI agents must perceive, reason, plan, and act in complex environments — often across multiple modalities, building on the vision-language models of the previous lecture.
- ▶ **Agentic AI** aims to build goal-directed, autonomous, and continuously learning systems on top of LLMs.
- ▶ The agent's most general interface to the world is **executable code** — so this lecture also covers how models write code, and how that ability is measured.
- ▶ The first agent frameworks (Auto-GPT, BabyAGI) opened new research frontiers in decision-making, robotics, and assistive AI — and new failure modes, which is why safety and control are part of the core material.

- ▶ Understand the principles of agentic AI and agentic loops.
- ▶ Describe how an LLM acts through function calling, and what MCP standardises.
- ▶ Differentiate between reactive and proactive agent behavior.
- ▶ Explain how code generation is evaluated (HumanEval, pass@k, SWE-bench, match-based metrics) and the methods behind modern code LMs (context engineering, infilling, retrieval, execution feedback).
- ▶ Describe continual learning challenges in agentic systems.
- ▶ Critically evaluate frameworks like Auto-GPT and BabyAGI.
- ▶ Reflect on safety, control, and ethical issues in agentic AI.

Agentic AI



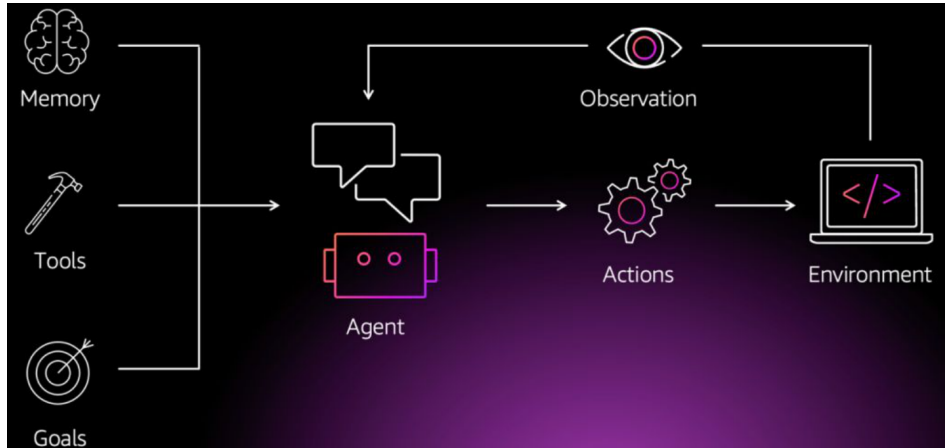
Source: Cadence, "What is Agentic AI?", https://www.cadence.com/en_US/home/explore/agentic-ai.html.

- ▷ AI systems that **perceive, reason, plan,** and **act** to fulfill goals.
- ▷ **Agent loop:** Observe → Plan → Act → Reflect → Learn
- ▷ Inspired by cognitive science, robotics, and autonomous agents.

Core Components:

- ▷ Memory
- ▷ Planning
- ▷ Tool-use (plugins, APIs)
- ▷ Feedback-driven behavior

What is Agentic AI?

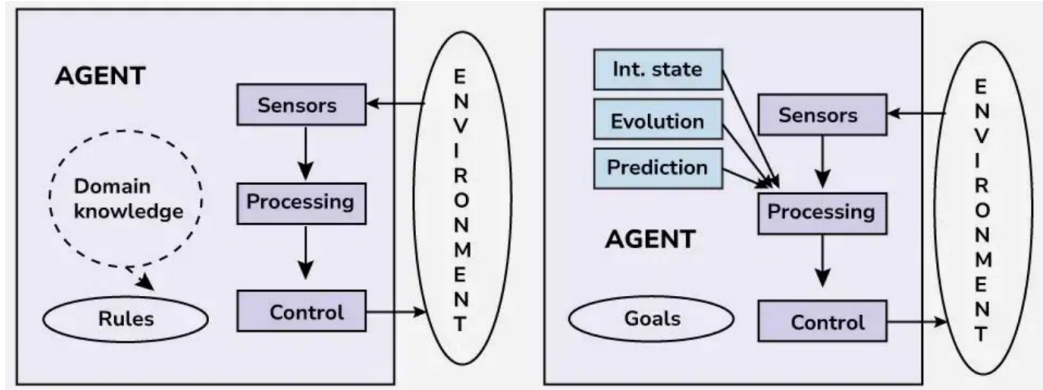


Source: Kipkemboi, Moura, Singh & Tsakpinis, "Build agentic systems with CrewAI and Amazon Bedrock", AWS Machine Learning Blog, 2025.

Feature	Reactive	Proactive
Response	Immediate	Goal-oriented
Planning	None	Yes
Learning	Event-triggered	Self-initiated
Example	Chatbots	Auto-GPT, Personal Assistants

Proactive agents generate goals, schedule actions, and evaluate results even without user prompts.

Reactive vs. Deliberative Agents



A reactive agent maps percepts to actions through fixed rules. A deliberative agent keeps internal state, predicts how the environment will evolve, and selects actions against a goal. Source: GeeksforGeeks, "Reactive vs Deliberative AI Agents".

- ▷ **Perceive:** Multimodal input (text, vision, memory)
- ▷ **Interpret:** NLP/vision models
- ▷ **Plan:** Task decomposition (e.g., LangChain chains)
- ▷ **Act:** API/tool calling
- ▷ **Reflect & Learn:** Refine memory or adjust goals

Applications:

- ▷ Personal AI assistants
- ▷ Research co-pilots
- ▷ Automated workflows

- ▶ **Function calling** is how an LLM acts: each tool is described to the model as a function with a name, a description, and a JSON schema for its parameters.
- ▶ The model does not execute anything. It emits a structured call (function name + arguments); the runtime executes it and feeds the result back as a new message.
- ▶ **The loop:** model receives the task → emits a call → runtime returns the result → model reasons over it → another call or a final answer. This is the ReAct pattern covered earlier, with the action step now carried by the API instead of by parsed text.
- ▶ Modern APIs support **parallel calls** (several independent tools in one step) and **structured outputs** (forcing the reply itself into a schema).

Tool definition given to the model:

```
{"name": "get_weather",  
  "description": "Current weather for a city",  
  "parameters": {"type": "object",  
    "properties": {"city": {"type": "string"}},  
    "required": ["city"]}}
```

User: "Do I need an umbrella in Jeddah today?"

Model -> call: get_weather(city="Jeddah")

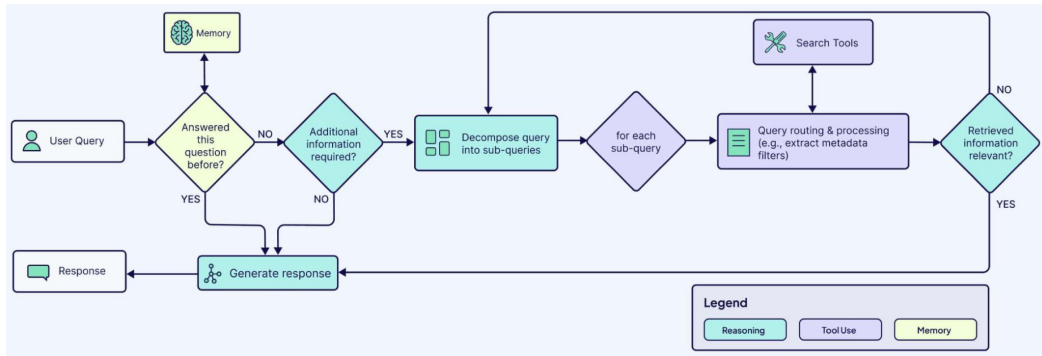
Runtime-> result: {"condition": "rain", "temp_c": 24}

Model -> "Yes, take one: it is raining in Jeddah (24 C)."

- ▶ **The problem.** Function calling describes tools *to one model, in one vendor's format*. Every new model or agent framework means rewriting the same integrations, so M agents times N tools costs $M \times N$ adapters.
- ▶ **Model Context Protocol (MCP)**, released by Anthropic in November 2024, is an open protocol for that connection, modelled on the Language Server Protocol used by code editors. It reduces the cost to $M + N$.
- ▶ **Three roles:** a **host** is the application the user talks to; a **client** lives inside the host and holds one connection; a **server** wraps a data source or service and advertises what it offers.
- ▶ A server exposes three kinds of thing: **tools** (actions the model can invoke), **resources** (data it can read), and **prompts** (templates the user can select). Messages travel as JSON-RPC.
- ▶ The payoff is that a filesystem, database, or issue-tracker server written once works with any MCP-capable agent, whoever built it.

Specification: <https://modelcontextprotocol.io/specification/latest>

- ▶ **Fixed RAG pipeline:** retrieve once for the raw query, then generate. Fails when the query is vague, multi-part, or needs information the first retrieval missed.
- ▶ **Agentic RAG** makes retrieval a tool the model controls:
 - **Decide when:** answer directly from parametric knowledge, or retrieve.
 - **Rewrite the query:** decompose multi-part questions into targeted searches.
 - **Iterate:** retrieve, read, notice what is missing, retrieve again.
 - **Judge the evidence:** discard irrelevant passages before answering.
- ▶ Costs more latency and tokens per query; pays off on multi-hop questions and heterogeneous document collections.



Source: Newhauser, Yadav, Monigatti & Çelik, "What Are Agentic Workflows? Patterns, Memory, Use Cases, and Examples", Weaviate blog, 2025.

Surveying (multimodal) LLM agents, Xie et al. sort every framework along two axes — **who plans** and **what is remembered**:

	Planner	Long-term memory
Type I	closed-source LLM, steered by prompts	none
Type II	open LLM, fine-tuned on action data	none
Type III	either	indirect : retrieved from an external store
Type IV	either	native : built into the planner itself

- ▷ Indirect memory is the Agentic RAG pattern from the previous section: experience lands in a vector store and returns through retrieval — Auto-GPT and BabyAGI (next) both live here.
- ▷ Multi-agent frameworks add a third axis: specialised agents dividing the work, with or without a *shared* long-term memory.

Xie et al., “Large Multimodal Agents: A Survey,” [arXiv:2402.15116](#).

Open-Source Agentic Frameworks

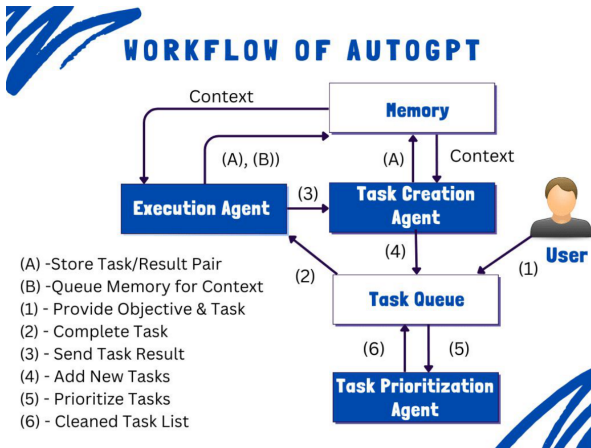
- ▷ **Open-source project** (Toran Bruce Richards, March 2023) that chained GPT-4 calls into a self-directed agent.
- ▷ Used **long-term memory** in a vector store, a file workspace, and self-generated tasks.
- ▷ **Requirements:**
 - Task input
 - Internet / API access
 - Feedback loop
- ▷ **Status:** the original agent now ships as classic/ and is unsupported; the project has become a visual builder, the AutoGPT Platform.

GitHub: <https://github.com/Significant-Gravitas/AutoGPT>

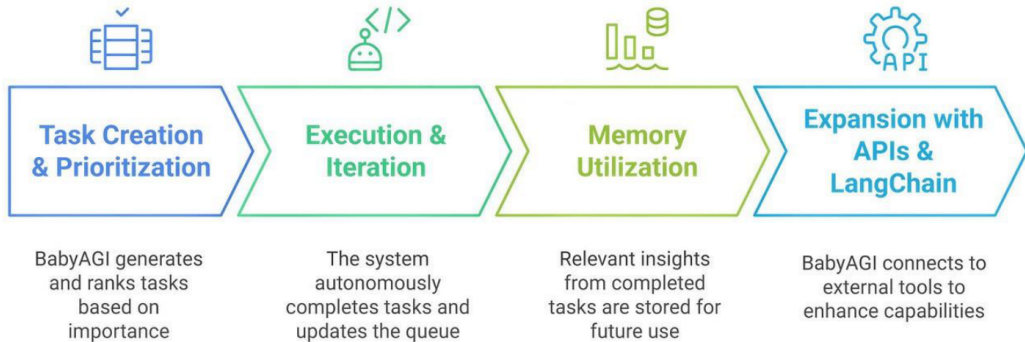
- ▶ **Lightweight Python agent loop** (Yohei Nakajima, March 2023), around 140 lines
- ▶ Generates tasks from a high-level objective
- ▶ Runs tasks, stores results in a vector store, and generates new tasks iteratively
- ▶ Emphasizes simplicity and minimalism

Original code: https://github.com/yoheinakajima/babyagi_archive

The babyagi repository itself now hosts a different, self-building agent framework.



Three LLM calls with one shared task queue and one memory store. Architecture after Y. Nakajima, "Task-Driven Autonomous Agent" (2023); this rendition is labelled "Workflow of AutoGPT" by its publisher, but the three named agents are BabyAGI's.



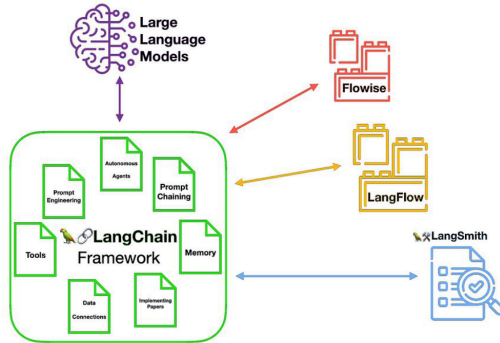
- ▶ **The measured results were poor.** GAIA asks 466 questions that are conceptually simple for a person but need tool use to answer. Human respondents score 92%; GPT-4 with plugins scores 15%. Auto-GPT on a GPT-4 backend manages 14.4% on the easiest tier and 0.4% on the middle tier, behind plain GPT-4 with hand-picked plugins.
- ▶ **Where the loops broke:** no reliable stopping criterion, so agents revisited the same subtask; errors compounded because nothing checked a step before the next built on it; and cost grew with every retry.
- ▶ AgentBench, across eight interactive environments, traces the gap to long-term reasoning, decision-making, and instruction-following rather than to the scaffolding around the model.
- ▶ **What survived:** the loop and its parts, a task queue, a memory store, tools behind a schema. What did not is the assumption that full autonomy from a one-line objective is the right target. Current systems narrow the task, verify each step, and keep a person in the loop for consequential actions.

Sources: Mialon et al., “GAIA”, [arXiv:2311.12983](#), Table 4; Liu et al., “AgentBench”, ICLR 2024, [arXiv:2308.03688](#).

- ▶ **LangChain:** Framework for chaining LLMs with tools, APIs, and memory; since version 1.0 (2025) its headline abstraction is agent construction on the LangGraph runtime.
- ▶ **CrewAI:** Python framework and hosted platform for orchestrating role-playing agents that collaborate on a task.
- ▶ **SuperAGI:** Dev-first open-source autonomous-agent framework with toolkits, agent memory, and telemetry. Largely dormant: last release January 2024.
- ▶ **OpenHands** (renamed from OpenDevin in August 2024): open platform for software-development agents, giving the agent a sandboxed shell, editor, and browser behind an event-stream architecture.

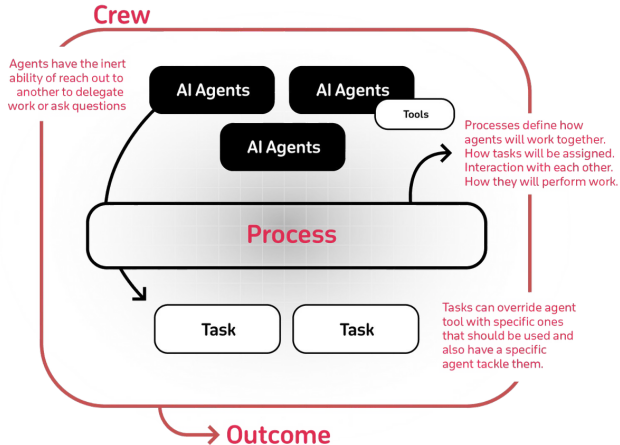


LangChain Ecosystem

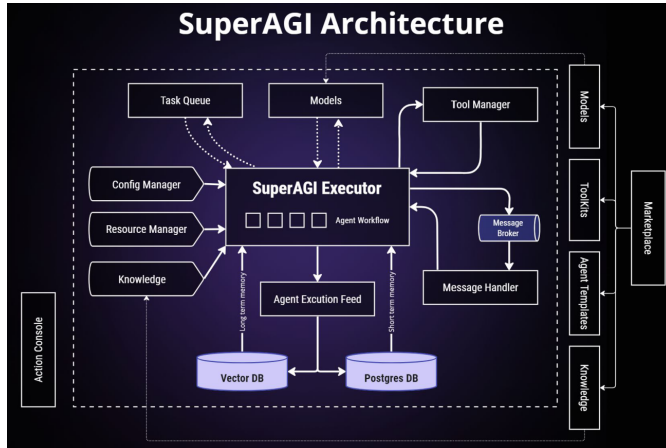


www.cobusgreyling.com

Source: C. Greyling, "The Growing LangChain Ecosystem".



A crew is a set of role-playing agents; a process defines how they hand work between each other. Source: CrewAI documentation, <https://docs.crewai.com>.

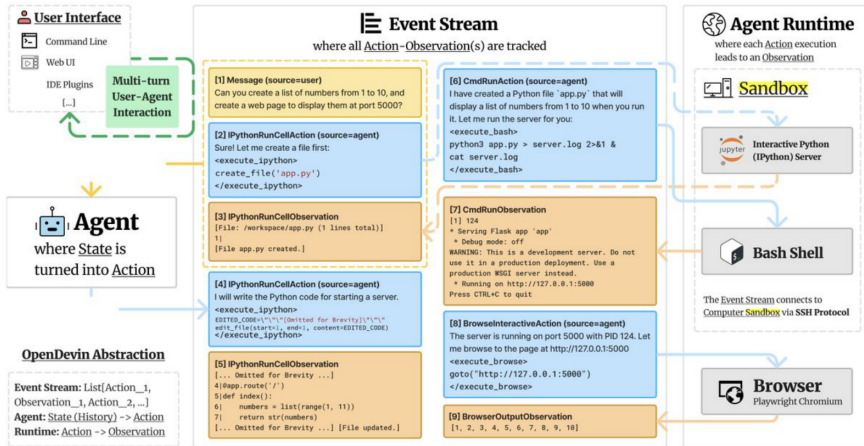


Source: SuperAGI project, <https://github.com/TransformerOptimus/SuperAGI>.

SuperAGI Core Features & Capabilities



Source: Servifyspheresolutions, "AutoGPT vs. BabyAGI vs. SuperAGI: Which AI Agent is Smarter for Autonomous Tasks?", Generative AI (Medium), 2025.



Every action and observation is appended to one event stream, which doubles as the agent's history and its memory. Source: Wang et al., "OpenHands: An Open Platform for AI Software Developers as Generalist Agents", ICLR 2025, [arXiv:2407.16741](https://arxiv.org/abs/2407.16741).

Code Generation

- ▶ **Generating executable code is how a model acts on the world at full generality.** Function calling (covered earlier) offers a fixed menu of tools; written code composes anything the runtime can do.
- ▶ The software-development agents seen earlier (OpenHands and its sandboxed shell) are built entirely on this ability.
- ▶ **Inputs and outputs:** natural language → code, surrounding code context → completion, even a screenshot of a page → the HTML behind it. The output is always text — but text a machine will run.
- ▶ **The economics are real:** in GitHub's controlled experiment, developers using Copilot finished the same implementation task 55.8% faster than the control group.

Peng et al., "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot," [arXiv:2302.06590](https://arxiv.org/abs/2302.06590).

Four properties separate code from prose, and each one shapes the methods that follow:

- ▶ **Strict grammar:** one wrong token is a syntax error, not a stylistic wobble — so sampling temperature is kept low, and decoding favours exactness over diversity.
- ▶ **Known semantic flow:** code parses into syntax trees and dataflow graphs — so metrics can compare structure, not just words.
- ▶ **Executable:** there is a ground truth a test suite can check — so evaluation and feedback loops can *run* the output instead of eyeballing it.
- ▶ **Written incrementally:** real programming edits the middle of existing files — so left-to-right completion alone is not enough.

- ▶ **HumanEval** (introduced with Codex): 164 hand-written Python problems — signature, docstring, examples — each judged by hidden unit tests. Hand-written precisely so the problems could not already sit in the training data.
- ▶ **The metric asks the right question:** not “does the code look right” but “if we sample k programs, does at least one pass the tests?”

$$\text{pass@}k = \mathbb{E}_{\text{problems}} \left[1 - \binom{n-c}{k} / \binom{n}{k} \right]$$

- ▶ Estimating from exactly k samples has high variance, so generate $n > k$ samples, count the c correct ones, and compute the expectation in closed form — the unbiased estimator above.
- ▶ The same pass@ k later became the exploration diagnostic for reasoning models (see the RL post-training lecture).

Benchmark	Year	Task	Judged by
CoNaLa	2018	StackOverflow question → snippet	text match
HumanEval	2021	docstring → function	unit tests
ODEX	2022	open-domain libraries	execution
ARCADE	2022	next notebook cell, in context	execution
SWE-bench	2023	GitHub issue → repository patch	repo's own tests
Design2Code	2024	webpage screenshot → HTML/CSS	visual similarity

- ▶ The trajectory: isolated snippet → real libraries → incremental context → whole repository → multimodal input.
- ▶ **SWE-bench** is the one agent evaluations now lean on: resolving a real issue takes navigation, long context, and precise edits — the full loop, not one completion.

Yin et al. [arXiv:1805.08949](#); Wang et al. [arXiv:2212.10481](#); Yin et al. [arXiv:2212.09248](#); Jimenez et al. [arXiv:2310.06770](#); Si et al. [arXiv:2403.03163](#).

- ▶ Code benchmarks live on GitHub; models train on GitHub. Sooner or later the test set is in the training set.
- ▶ **The symptom:** ARCADE found models noticeably weaker on newly written notebooks than on notebooks mined from the web; several code LMs post strong HumanEval numbers yet stumble on problems published after their training cutoff.
- ▶ **The fix is a moving target:** LiveCodeBench harvests fresh contest problems continuously and tags each with its release date, so a model is only scored on problems newer than its training data.
- ▶ Same discipline as the train/test hygiene taught in the preprocessing lectures — applied to the whole internet.

Jain et al., “LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code,”

[arXiv:2403.07974](https://arxiv.org/abs/2403.07974).

- ▶ Execution-based scoring needs unit tests and a safe sandbox — rarely available inside large codebases — and it is blind to style and maintainability.
- ▶ **BLEU** (from the sequence-to-sequence lecture) treats code as text: n -gram overlap with a reference solution.
- ▶ **CodeBLEU** adds what code uniquely offers: a syntax-tree match and a dataflow match on top of the n -grams, so renaming a variable consistently is no longer punished like a wrong word.
- ▶ **CodeBERTScore** drops string matching entirely and compares embeddings from a code-pretrained encoder — the best correlation with both human judgement and execution accuracy among the match-based metrics.

Ren et al., “CodeBLEU,” [arXiv:2009.10297](https://arxiv.org/abs/2009.10297); Zhou et al., “CodeBERTScore,” [arXiv:2302.05527](https://arxiv.org/abs/2302.05527).

- ▷ The base method is nothing exotic: feed the preceding code to a language model and let it continue. Every serious LLM is now trained on code.
- ▷ What separates a demo from a product is **what goes into the prompt**. Reverse-engineering GitHub Copilot showed a careful assembly line:
 - language and relative path of the current file, as metadata;
 - the text before and after the cursor;
 - similar snippets pulled from the **20 most recently viewed files** in the same language;
 - imported files, when they fit.
- ▷ In other words: production code completion is prompt engineering at industrial scale — the same discipline as the prompting lecture, applied per keystroke.

P. Thakkar, "Copilot Internals," 2023,

<https://thakkarparth007.github.io/copilot-explorer/posts/copilot-internals>.

- ▶ Autoregressive completion only appends — but programmers mostly edit the *middle* of files: the cursor sits between a prefix and a suffix.
- ▶ **Fill-in-the-middle training:** cut a document into (prefix, middle, suffix), move the middle to the end, and train the model to produce it from the surrounding pieces — causal training that yields bidirectional conditioning at inference.
- ▶ Introduced for code by InCoder; now a standard ingredient — StarCoder 2 and Code Llama both train with infilling objectives so the same checkpoint can complete and repair.

Fried et al., “InCoder: A Generative Model for Code Infilling and Synthesis,” [arXiv:2204.05999](https://arxiv.org/abs/2204.05999).

- ▶ **Retrieval-augmented generation, for code:** retrieve similar snippets, or better, the *documentation* of the library being used — DocPrompting shows retrieved docs let a model call libraries it never saw in training. Same RAG machinery as the prompting lecture, different corpus.
- ▶ **Execution feedback:** code can be run, so run it — sample many candidates, execute them, and keep the ones that agree or pass the visible tests (AlphaCode's filter, MBR-EXEC's agreement vote).
- ▶ **Repair from error messages:** feed the traceback back in and let the model fix its own output; InterCode formalizes this as an interactive environment where execution output is the observation.
- ▶ Observe → act → read the result → act again: **the agent loop from earlier**, specialised to code.

Zhou et al., "DocPrompting," [arXiv:2207.05987](#); Shi et al., [arXiv:2204.11454](#); Yang et al., "InterCode," [arXiv:2306.14898](#).

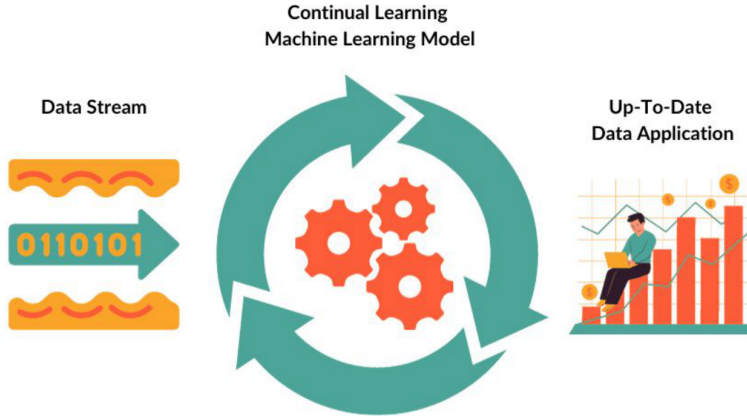
Model	Recipe	What it added
Codex (2021)	GPT-3 continued on GitHub	powered the original Copilot; HumanEval
Code Llama (2023)	Llama 2 continued on code	long-context RoPE; infilling; synthetic instructions
StarCoder 2 (2024)	from scratch on The Stack v2	license-aware data; issues, notebooks, IRs
DeepSeek Coder (2024)	from scratch, 87% source code	repo-level, dependency-ordered training files

- ▶ Architecturally they are all Llama-style decoders; the differences that matter are in the **data**: what code, how deduplicated, how ordered, and whether licenses and opt-outs are honoured (The Stack v2's defining feature).
- ▶ Metadata tricks pay off: StarCoder prepends repository and file names half the time, so at inference the same tags can steer the model.
- ▶ Since then the specialist/generalist line has blurred: every frontier model trains heavily on code (recall LLaMA 3's HumanEval scores in the Recent Advancements lecture).

Chen et al. [arXiv:2107.03374](https://arxiv.org/abs/2107.03374); Rozière et al. [arXiv:2308.12950](https://arxiv.org/abs/2308.12950); Lozhkov et al. [arXiv:2402.19173](https://arxiv.org/abs/2402.19173); Guo et al. [arXiv:2401.14196](https://arxiv.org/abs/2401.14196).

- ▶ **The benchmark moved to the agent:** SWE-bench hands the model a live repository and a real issue; solving it takes the whole loop — explore, edit, run the tests, read the failures, edit again.
- ▶ **The harness exists:** OpenHands (covered earlier) is exactly this loop productised — a sandboxed shell, editor, and browser behind an event stream.
- ▶ **The reward is verifiable:** a passing test suite is a rule-based reward no human labeller has to provide — which is why code sits beside math at the heart of RL post-training (see the GRPO lecture).
- ▶ Code is the domain where feedback is cheapest, fastest, and most trustworthy. That is why agents learned to program before they learned to do almost anything else.

Continual Learning in Agents



Source: N. Van Otten, "Continual Learning Made Simple, How To Get Started & Top 4 Models", Spot Intelligence, 2023.

▷ **Agentic AI should evolve with time:**

- Learn from feedback
- Adapt to new tasks
- Retain useful knowledge

▷ **Real-world scenarios:**

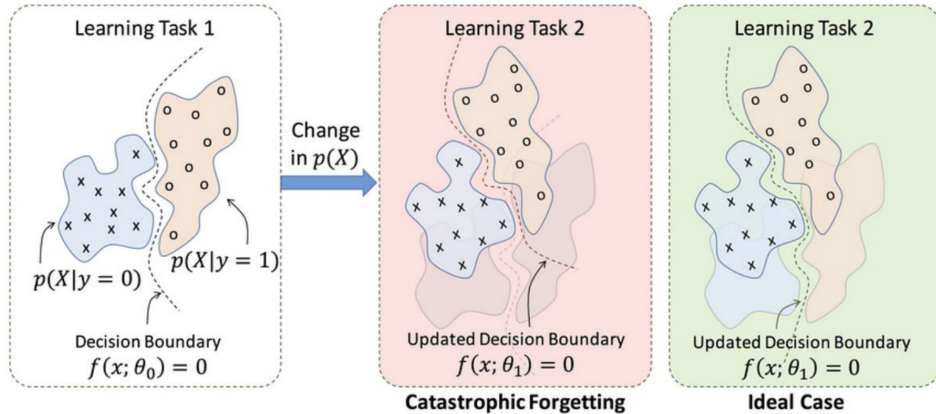
- **Personalization:** the same assistant serving one user for a year.
- **Environment drift:** tools, APIs, and documents change under the agent.
- **Avoiding forgetting:** new skills must not erase old ones.

- ▶ **Catastrophic forgetting:** Agents may lose previously acquired knowledge when learning new tasks.
- ▶ **Stability-plasticity dilemma:** Balancing the ability to learn new information (plasticity) with retaining old knowledge (stability).
- ▶ **Modality drift:** Changes in the types or distributions of input modalities over time.
- ▶ **Scaling memory:** Managing both semantic (general knowledge) and episodic (specific experiences) memory as agents interact with the world.

Solutions:

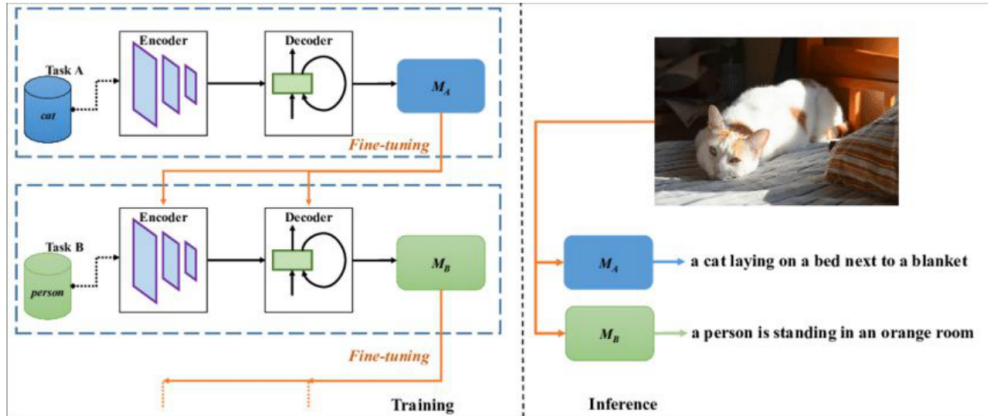
- ▶ **Experience replay:** interleave a buffer of past examples with the new ones, so old tasks keep contributing gradient.
- ▶ **Progressive networks:** freeze the trained columns and add a fresh one per task, with lateral connections, so nothing is overwritten.
- ▶ **Elastic weight consolidation (EWC):** penalise moving the weights that mattered most for earlier tasks.

Catastrophic forgetting



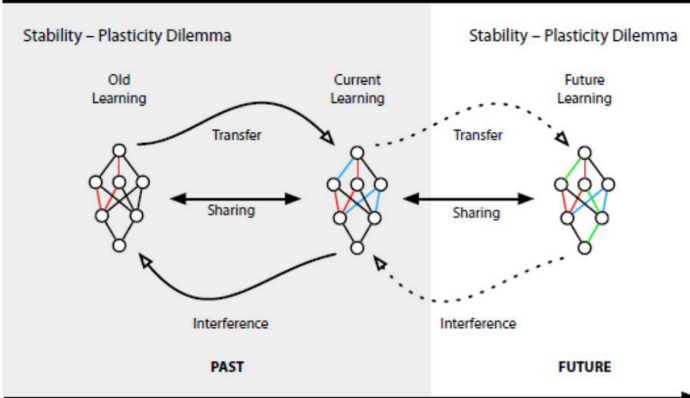
Source: Kolouri, Ketz, Zou, Krichmar & Pilly, "Attention-Based Selective Plasticity", [arXiv:1903.06070](https://arxiv.org/abs/1903.06070), Figure 1.

Catastrophic forgetting

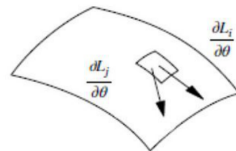


After fine-tuning on task B, the same cat photograph is captioned “a person is standing in an orange room”. Source: Nguyen et al., “ContCap: A scalable framework for continual image captioning”, [arXiv:1909.08745](https://arxiv.org/abs/1909.08745).

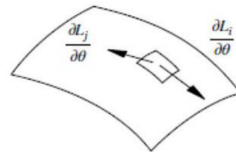
A. Transfer – Interference Trade-off



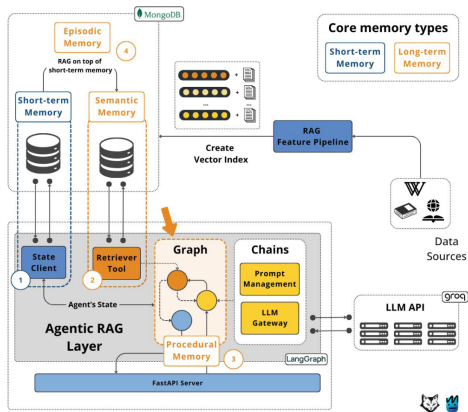
B. Transfer



C. Interference



Gradients that point the same way across tasks give transfer; gradients that oppose give interference. Source: Riemer et al., “Learning to Learn without Forgetting by Maximizing Transfer and Minimizing Interference”, ICLR 2019, [arXiv:1810.11910](https://arxiv.org/abs/1810.11910), Figure 1.



Source: P. Iusztin, "Memory: The Secret Sauce of AI Agents", Decoding AI, 2025.

Safety, Ethics, and Control

▷ Risks:

- Autonomous actions without human oversight
- Errors in tool or API use
- Prompt injection or goal hijacking
- Recursive self-improvement without constraints

▷ Mitigations:

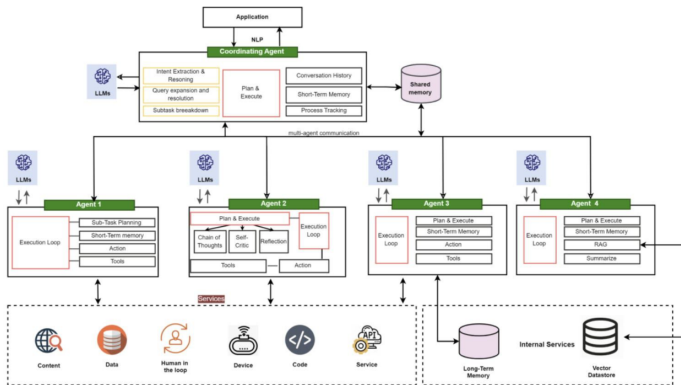
- Approval-based execution: a person confirms a consequential action before the runtime executes it
- Guardrails: separate classifiers that screen the agent's inputs and outputs for injection attempts and policy violations
- Explainability (XAI) for agent decisions

Each of these, and the threat model behind it, is treated in full later in the course.

Governance Concerns Reported by Technology Leaders

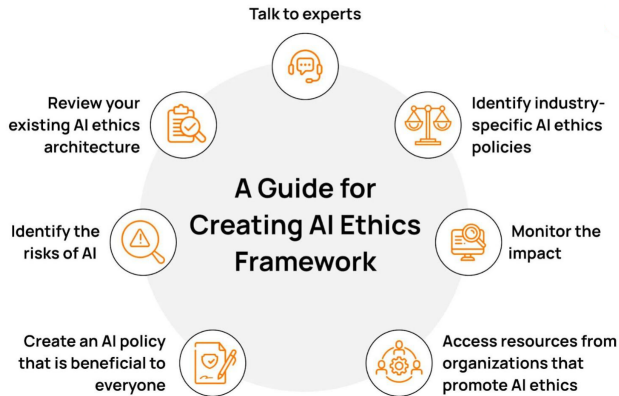


Source: I. Katusic, "Top 7 Concerns of Technology Leaders That Implemented Agentic AI", CTO Academy, 2025.



An example multi-agent architecture built from specialised agents. Specialisation is one agentic pattern among several, and which pattern fits depends on the use case.

Source: OWASP GenAI Security Project, "Agentic AI – Threats and Mitigations", Agentic Security Initiative, v1.1 (December 2025), p. 10.



Source: P. Parsani, "How to Create an AI Ethics Framework for your Company?", Cut the SaaS, 2024.

- ▶ **Alignment:** Are agents acting in users' interests?
- ▶ **Accountability:** Who is responsible for AI actions?
- ▶ **Control:** Can humans override or audit decisions?

Key Areas:

- ▶ Fairness in multimodal data
- ▶ Bias amplification (vision + language)
- ▶ Transparency in decision-making

Future Directions

- ▶ **Multimodal grounding in real-world environments:** How can agents connect language, vision, and action in dynamic, noisy settings? SayCan is one line of attack: score what the language model *wants* to do against what the robot can actually do here.
- ▶ **World model integration:** Give the agent a learned internal model of how its environment evolves, so it can plan against predicted states rather than only react to observed ones.
- ▶ **Agent collaboration:** Enabling effective communication and coordination in multi-agent systems.
- ▶ **Scaling memory-efficient agents:** Designing architectures that scale to long-term, large-scale memory without prohibitive costs.
- ▶ **Robust reward models:** Developing reliable reward and feedback mechanisms for open-ended, real-world tasks.
- ▶ **Meta-cognition:** Building agents that can introspect, self-monitor, and adapt their own reasoning processes.

- ▶ **Agentic AI** adds goals, memory, and tools to a language model. **Function calling** is the mechanism; **MCP** standardises the interface.
- ▶ **The first autonomous frameworks (Auto-GPT, BabyAGI)** established the loop but performed poorly on real tasks. Their components survived; the assumption of full autonomy did not.
- ▶ **Code generation** is the agent's strongest actuator: executability gives cheap, trustworthy feedback — unit-test evaluation (pass@k, SWE-bench), execution-driven repair, and verifiable rewards all flow from it.
- ▶ **Continual learning** remains open: agents must absorb new tasks without overwriting old ones.
- ▶ Agency turns a wrong answer into a wrong action, which is why safety and oversight get their own treatment next.

- [1] Yao, S., Zhao, J., Yu, D., et al. (2023).
ReAct: Synergizing Reasoning and Acting in Language Models.
ICLR 2023. [arXiv:2210.03629](https://arxiv.org/abs/2210.03629)
- [2] Anthropic (2024).
Model Context Protocol: specification and SDKs.
<https://modelcontextprotocol.io/specification/latest>
- [3] Richards, T. B. (Torantulino) et al. (2023).
AutoGPT.
<https://github.com/Significant-Gravitas/AutoGPT>
- [4] Nakajima, Y. (2023).
BabyAGI (original task-driven agent; archived 2024).
https://github.com/yoheinakajima/babyagi_archive
- [5] Wang, X., Li, B., Song, Y., et al. (2025).
OpenHands: An Open Platform for AI Software Developers as Generalist Agents.
ICLR 2025. [arXiv:2407.16741](https://arxiv.org/abs/2407.16741)

- [6] Mialon, G., Fourrier, C., Swift, C., Wolf, T., LeCun, Y., Scialom, T. (2023).
GAIA: a Benchmark for General AI Assistants.
[arXiv:2311.12983](#)
- [7] Liu, X., Yu, H., Zhang, H., et al. (2024).
AgentBench: Evaluating LLMs as Agents.
ICLR 2024. [arXiv:2308.03688](#)
- [8] Ahn, M., Brohan, A., Brown, N., Chebotar, Y., et al. (2022).
Do As I Can, Not As I Say: Grounding Language in Robotic Affordances.
CoRL 2022, PMLR 205:287–318. [arXiv:2204.01691](#)
- [9] Xie, J., Chen, Z., Zhang, R., et al. (2024).
Large Multimodal Agents: A Survey.
[arXiv:2402.15116](#)
- [10] Chen, M., Tworek, J., Jun, H., et al. (2021).
Evaluating Large Language Models Trained on Code (Codex, HumanEval, pass@k).
[arXiv:2107.03374](#)

- [11] Jimenez, C., Yang, J., Wettig, A., et al. (2024).
SWE-bench: Can Language Models Resolve Real-World GitHub Issues?
ICLR 2024. [arXiv:2310.06770](#)
- [12] Jain, N., Han, K., Gu, A., et al. (2024).
LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code.
[arXiv:2403.07974](#)
- [13] Ren, S., Guo, D., Lu, S., et al. (2020).
CodeBLEU: a Method for Automatic Evaluation of Code Synthesis.
[arXiv:2009.10297](#)
- [14] Zhou, S., Alon, U., Agarwal, S., Neubig, G. (2023).
CodeBERTScore: Evaluating Code Generation with Pretrained Models of Code.
EMNLP 2023. [arXiv:2302.05527](#)
- [15] Fried, D., Aghajanyan, A., Lin, J., et al. (2022).
InCoder: A Generative Model for Code Infilling and Synthesis.
ICLR 2023. [arXiv:2204.05999](#)

- [16] Zhou, S., Alon, U., Xu, F., Wang, Z., Jiang, Z., Neubig, G. (2022).
DocPrompting: Generating Code by Retrieving the Docs.
ICLR 2023. [arXiv:2207.05987](#)
- [17] Yang, J., Prabhakar, A., Narasimhan, K., Yao, S. (2023).
InterCode: Standardizing and Benchmarking Interactive Coding with Execution Feedback.
NeurIPS 2023. [arXiv:2306.14898](#)
- [18] Rozière, B., Gehring, J., Gloeckle, F., et al. (2023).
Code Llama: Open Foundation Models for Code.
[arXiv:2308.12950](#)
- [19] Lozhkov, A., Li, R., Allal, L. B., et al. (2024).
StarCoder 2 and The Stack v2: The Next Generation.
[arXiv:2402.19173](#)
- [20] Guo, D., Zhu, Q., Yang, D., et al. (2024).
DeepSeek-Coder: When the Large Language Model Meets Programming.
[arXiv:2401.14196](#)

- [21] Peng, S., Kalliamvakou, E., Cihon, P., Demirer, M. (2023).
The Impact of AI on Developer Productivity: Evidence from GitHub Copilot.
[arXiv:2302.06590](https://arxiv.org/abs/2302.06590)

- [22] Thakkar, P. (2023).
Copilot Internals (reverse engineering the Copilot prompt).
thakkarparth007.github.io/copilot-explorer